



Addis Ababa University
College of Natural and Computational Sciences
Department of Computer Science
Programming and Algorithms Module

Part I: Computer Programming



Part II: Object Oriented Programming

Part III: Design and Analysis of Algorithms

Part IV: Data Structure and Algorithms

May 2024
Addis Ababa,
Ethiopia

Chapter One: Object Oriented Programming

1.1. Overview

Object oriented programming: allow the programmer to break down the problem into objects.

Objects: self-contained entities consisting of both data and operations together.

E.g. java, C++, C#, VB6, Python...

OOP: is a programming language model organized around objects rather than "actions" and data rather than "logic".

Java technology consists of an OO programming language and a platform on which programs can be run.

The Java compiler translates the program into an intermediate language called **Java bytecodes**.

Java bytecodes are platform-independent.

Guaranteed to be Write Once, Run Anywhere.

There are different Java interpreters for different platforms.

Java is: + Object Oriented + Platform Independent

+ Simple + Secure

+ Portable +

Multithreaded + Robust

Q1: What is the d/c between compiler and interpreter?

- The Java platform is a bit different to other platforms, in that it consists of software only – that means it is hardware-independent.

- The Java **platform** consists of:

- ❖ the Java VM
- ❖ the Java API (Application Programming Interface).

- The API provides libraries, which groups a number of related classes and interfaces together into what is called a Java **package**.

1.2. Fundamentals of Objects and Classes

Programming Paradigms:

1. Procedural Programming: (remember)

- is a list of step-by-step instructions
- usually having a linear order of execution.
- Includes C, C++, FORTRAN, Pascal, and Basic.

2. Object oriented programming:

- allow the programmer to break down the problem into *objects*:
- Easy to debugging.
- E.g. java, C++, C#, VB6, Python

3. Declarative:

- Describe a problem rather than defining a solution
- Describes *what* something is like, rather than *how* to create it.

̄. Objects: (Recall)

- Are instances of a class

- Are the basic runtime entities in an OO program.
- Are identifiable things or individuals?
- Are of significance to the system
- Have states reflected in the attributes (data) held about the object
- Have characteristic behaviour (i.e. functionality)

g. Class:

- Is a blueprint for an object.
- is the *definition of a set of objects*
- which are all similar in terms of their attributes
- A class consists of a class name, data and methods.
- e.g. customer class defines all customer objects.

h. Members:

- Collectively, the methods and variables defined within a class are called members of the class.
- These are local, instance and Class/static variable

i. Abstraction:

- provides only essential information to the outside world.
- Or hiding their background details, i.e., to represent the needed information in program without presenting the details.
- For example, a bank system.

j. Encapsulation:

- means the wrapping up of data and methods into a single unit (a class).
- It also means that the data inside a class is hidden from everything outside the class.
- The data can only be accessed by invoking the methods of the class.
- It is placing the data and the functions that work on that data in the same place.

k. Inheritance:

- It is the way in which objects of one class get the properties of objects of another class(data & methods).
- For example, Person, employee and retiree
- Inheritance provides the idea of reusability – the Person class can be reused without making changes to it, because another class can be made to inherit from it.
- Types of Inheritance? read

l. Polymorphism:

- The word polymorphism means the ability to take more than one form.
- In terms of the OOP, this means that a particular operation may behave differently for different sub-classes of the same class.

Components of java program

Java program = **Comments** + **Token(s)** + **white space**

1. **Comments:** Java supports *three* types of *comment* delimiters. A

single line comment `//`, 2 multiline comments `/* .... */` and `/** ....*/`.

The last one is used by javaDoc

2. **Token** is the *smaller* individual units inside a program the compiler recognizes when building up the program.

➤ There are **five** types of **Tokens** in Java:

- ✓ **Reserved keywords:** class, extends, implements, for...
- ✓ **Identifiers:** programmers given name; grade, gRade...
- ✓ **Literals (Constants values):** 55, 60.78, "Hi", 'M'... Java language specifies **five** major types of literals => integer, floating, character, string & Boolean literals
- ✓ **Operators:** Are symbols that take one or more arguments (operands) and operates on them to produce a result. An operator performs a function on one (Unary), two (Binary), or three operands (Ternary). Java has one ternary operator, `?:`, which is a short-hand *if-else* statement. E.g. `c=x?w:s;` Operators can be Arithmetic, Logical, assignment, Relational, conditional, Increment,
- ✓ **Separators:** symbols used to indicate where groups of codes are divided and arranged. E.g. `() , {} , [] , ; , , , .`

3. **Whitespace** in Java is used to separate the tokens in a Java source file. It is required in some places, such as between access modifiers, type names and Identifiers, and is used to improve readability elsewhere.

Java **whitespace** consists of the

- a. Space character `' '` (0x20),
- b. The tab character (hex 0x09), `(\t)`
- c. The line separators characters `newline(\n)` (hex 0x0a)
- d. The carriage return (hex 0x0d) characters `(\r)`
- e. The form feed character (hex 0x0c), `(\f)` : produce ↑

1.3. Variables in Java

Variable = Data Type + Identifier

There are **three** kinds of variables in Java:

A. Local variables:

- Local variables are declared in methods, constructors, or blocks.
- has only local scope
- Access modifiers cannot be used for local variables.
- They are visible only within the declared method, constructor, or block.
- Have not a default value
- *Should be declared and initialized @ z first time.*
- Declare before use anywhere in a method or block.

- Other methods in the class aren't even aware that the variable exists.

E.g.

```
if(x > 100) { String s1 = "Hello"; }
```

```
String a=s1 + "Java" // Is this valid?
```

- You cannot use s1 outside of that **if** block.
- Created when method or constructor is entered.
- Destroyed on exit.

B. Instance variables:

- Are declared in a class, but outside a method, constructor or any block.
- Have default values.
- are variables which are bound to the object itself. **Meaning**=>
- Every instance of that class (object) has **it's own copy** of that variable
- Are created when an object is created with the use of the keyword 'new' and destroyed when the object is destroyed.
- Can be declared in class level *before or after use*.
- Access modifiers can be given for instance variables.
- Are visible for all methods, constructors and block in the class.

E.g.

```
class MyClass { float aFloat; }
```

NB: By default, a member declared within a class is an instance member.

C. Class/Static variables

- Are declared with the **static** keyword in a class, but outside a method, constructor or a block.
- There would only be **one copy** of each class variable per class, regardless of how many objects are created from it.
- Are rarely used other than being declared as constants. (public static final)
- Are stored in the static memory.
- Static variables are created when the program starts and destroyed when the program stops.
- Visibility is similar to instance variables. However, most static variables are declared public
- Default values are same as instance variables.
- Static variables can be accessed by calling with the class name `ClassName.VariableName`.
- When declaring class variables as **public static final**, then variable names (constants) are all in upper case.

Recall Static Methods

Data Type

- It refer to an *extensive system* used for declaring variables or functions of different types.
- The type of a variable determines how much space it occupies in

storage and how the bit pattern stored is interpreted.

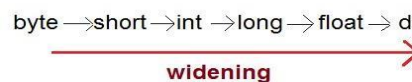
Java has two basic data types

1. **Primitive Data Types:** are 8 in java (byte, short, int, long, float, double, char & Boolean)
2. **Reference types:** Arrays, classes, and interfaces, String)

- ☞ **Note:** Boolean type has a value true or false (there is no conversion b/n Boolean and other types).
- ☞ All *non-primitive* types are reference types, so all class types are reference types.
- ☞ All integers in Java are **signed**, Java doesn't support **unsigned** integers.

Type casting

- Assigning a value of one type to a variable of another type is known as Type Casting.
- In Java, type casting is classified into two types
 1. **Widening, (Implicit)** or Automatic Casting: Java supports automatic widening, a kind of conversion from one type to another which doesn't lose information.



Two conditions for automatic casting

- ✓ Types are compatible
- ✓ Destination is wider than the source

Widening conversion usually occur automatically.

Example: `int f = 9;`
`float g = f;`

`long l=560; float f = 1;` //no explicit type casting required

2. **Narrowing (Explicit) Casting:** if you want to narrow or go in the other direction, Java won't do that automatically. You have to do that programmatically.



Syntax: `smallerDT`
`var=(smallerDT)largerDTVar`
OR `largerValue;`

Example: `float y=78.9f;` `byte b = (byte)y;` //explicit type casting required

`int x = (int)2.7;` `double d = 100.04;` `long l = (long)d;`

Question: What is the merit and demerit of implicit and explicit type casting?

Boxing and Unboxing:

- ☞ Autoboxing is the automatic conversion that the Java compiler makes between the primitive types and their corresponding *object wrapper classes*.
- ☞ For example, converting an int to an Integer, a double to a Double, and so on.
- ☞ If the conversion goes the other way, this is called unboxing.

- ☞ Converting primitive data types into *object* is called *boxing*.
- ☞ Therefore, while using a wrapper class you just need to pass the value of the primitive data type to the constructor of the Wrapper class.
- ☞ And the *Wrapper object* will be *converted back to a primitive data type*, called *unboxing*.
- ☞ The Number class is part of the *java.lang* package.

Overview of Java statements

- ☞ Statements are roughly equivalent to sentences in natural languages.
- ☞ A statement forms a complete unit of execution.

There are **three** kinds of **statements**:

1. **Expression statements**: Perform *computations* and *return* values. It is a series of variables, operators, and method calls that evaluates to a single value. (constructed according to the syntax of the language). There are:
 - A. Assignment expressions,
 - C. Any use of ++ or -
-,
 - B. Method calls,
 - D. Object creation expressions ,
2. **Declaration statements**: declares/
Introduces a very first variables E.g.
double c = 8.4;
3. **Control flow statements**: regulates the order of statements execution

Selection: the **if()** and **switch()** statements

Iteration: The **for()**, **while()** and **do...while()** statements

4. **Jumping Statements**: to jump out of loops and to control other areas of program java uses **break** and **continue** statements.

The **break** statement: used to exit a loop

The **Continue** Statement: used to force program to go back to the top of a loop

Labels: Java does **not** include a **goto** statement.

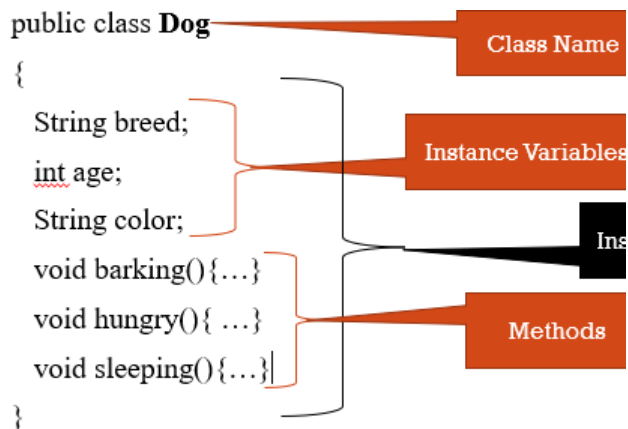
Instead of **goto**, Java allows you to combine break and continue with a label.

Chapter Two: Classes and Objects

2.1. Fundamentals of Classes

- ☞ A class is a blue print from which individual objects are created.
- ☞ Class is a template for an object, and an object is an instance of a class
- ☞ Class is the logical construct upon which the **entire** Java language is built because it defines the shape and nature of an object.
- ☞ As such, the class forms the basis for object-oriented programming in Java.
- ☞ Classes usually consist of two things: *instance variables* and *methods*
- ☞ Any concept you wish to implement in a Java program must be *encapsulated within a class*.
- ☞ A sample of a class is given below:

```
public class Dog
{
    String breed;
    int age;
    String color;
    void barking(){...}
    void hungry(){ ...}
    void sleeping(){...}
}
```



- ☞ The code is contained within methods (The operation is done inside)
- ☞ Thus, the data for one object is separate and unique from the data for another.

- ☞ All methods have the same general form as main()
- ☞ However, most methods will not be specified as static or public.
- ☞ Java classes do not need to have a main() method except the starting point for your program.
- ☞ Further, applets don't require a main() method at all.
- ☞ A class defines a new type of data (it is a reference type).
- ☞ In this case, the new data type is called **Dog**.
- ☞ You will use this name to declare objects of type **Dog**.
- ☞ a class declaration only creates a *template*; it does not create an actual object.
- ☞ To actually create a **Dog** object, use the second statement.

- ❖ **Dog myDog;** // Creates only the template
- ❖ **myDog = new Dog();** // create a Dog object called myDog
- ❖ After this statement executes, myDog will be an instance of Dog.
- ❖ Thus, it will have “physical” reality .

2.2. Types of classes

U. The Common types of classes:

1. **Wrapper Class:-** The eight classes of java.lang package

2. **Abstract Class:-** A classes declared with abstract keyword and *can't be instantiated*.
3. **Final Class:-** A class declared with final keyword and *can't be inherited*.
4. **Input-Output Class:-** Available in java.io package. contains all the required classes to perform I/O operations
5. **String Class:-** Immutable Class created to manipulate strings(Sequence of chars).
6. **System Class:-** Contains several useful class fields and methods. It cannot be instantiated. E.g. standard input, standard output, and error output streams
7. **Mutable Class:-** Have fields that can be changed.

Λ. According to access modifiers: Classes are classified in to 4:

1. **Public Classes:** Accessible to the world
2. **Private Class:** Used only inside a block(e.g with in the outer class)
3. **Protected Class:** Used in parent class, package and other subclasses
4. **Default Class:** Used only in parent class and package

h. According to Inheritance: Classes are classified in to 2:

1. **Supper Class:** The class that is inherited.
2. **Sub Class:** The class that does the inheriting.

• Based on the source file they are located classes are *classified in to two*.

1. **Internal classes:** Located at the starting pointing file, includes main().
2. **External classes:** Located in a separate source file (outside the starting pointing file).

2.3. Source file declaration rules

☞ These rules are essential when declaring classes, import statements and package statements in a source file.

1. There can be only one public class per source file.
2. A source file can have multiple nonpublic classes.
3. The **public class** name should be the name of the source file as well which should be appended by .java at the end.

For example: the class name is public class Employee{} then the source file should be as Employee.java.

☞ If the class is defined inside a package, then the package statement should be the first statement in the source file.

☞ If import statements are present then they must be written between the

package statement and the class declaration.

- ☞ If there are no package statements then the import statement should be the first line in the source file.
- ☞ Import and package statements will imply to all the classes present in the source file.
- ☞ It is not possible to declare different import and/or package statements to different classes in the source file.

2.4. Instantiating and initializing class objects

- ☞ Every Dog object will contain its own copies of the instance variables breed, age, and color.
- ☞ To access these variables, you will use the dot (.) operator.
- ☞ The dot operator links the name of the object with the name of an instance variable.
- ☞ For example, to assign the age variable of myDog the value 10, you would use the following statement:

`myDog.age = 10; //This is called
instantiating a class.`

- ☞ This statement tells the compiler to assign the copy of age that is contained within the myDog object the value of 10, breed to Australian cattle dog, and color of red....

```
class Dog {  
    int age; String breed, color; }
```

```
class DogDemo { // This class declares an  
    object of type Dog.  
    public static void main(String args[]) {  
        Dog myDog = new Dog();  
        String doginfo; //used to hold new Dog's  
        attribute  
        myDog.age = 10;  
        myDog.breed="Australian Cattel Dog";  
        myDog.color = "Red"; //assign values to  
        doginfo= " Breed: " + myDog.breed +  
        "Dog's age:" + myDog.age + "Color: " +  
        myDog.color;  
        System.out.println("The dog's " + doginfo);  
    } }
```

Declaring Objects

- When you create a class, you are creating a new data type.
- You can use this type to declare objects of that type.
- the following is used to declare an object of type Box:
 - ❖ `Dog myDog = new Dog();`
OR
 - ❖ `Dog myDog; // declare
reference/template to object`
 - ❖ `myDog = new Dog(); //
allocate a Dog object`

Methods

- Are members of classes and do operations.
- Allows different classes to communicate each other.
- E.g. `type name(parameter-
list)`
 `{ // body of method`
 `//...`
 `return value; // if`
 `type is not void`

```
}
```

Read About Methods...Returning a Value

Methods...Adding a method that takes parameters

Method Overloading

- Is a process of having the same name for different methods as long as their parameter list or data types in the parameter list is different?
- The return type alone is insufficient to distinguish two versions of a method.
- Parameters are key identifiers of OM.
- Is one of the ways that Java supports polymorphism
- Java's automatic type conversions can play a role in overload resolution.
- Example: // Demonstrate method overloading.

```
class OverloadDemo {  
void test() {  
System.out.println("No  
parameters"); }  
void test(int a) { // Overload  
test for one integer parameter.  
System.out.println("a: " + a); }  
void test(int a, int b) { //  
Overload test for two integer  
parameters.  
System.out.println("a and b: " +  
a + " " + b); }  
double test(double a) { //  
overload test for a double  
parameter  
System.out.println("double a: "  
+ a);  
return a*a; } }
```

```
class OverLoad {  
public static void main(String  
args[]) {  
OverloadDemo ob = new  
OverloadDemo();  
double result; // call all  
versions of test()  
ob.test(); ob.test(10);  
ob.test(10, 20);  
result = ob.test(123.25);  
System.out.println("Result of  
ob.test(123.25): " + result); }  
}
```

Method Overriding

In a class hierarchy, when a method in a subclass has the same name and type signature as a method in its superclass, this is called method Overriding.

- the method in the subclass is said to override the method in the superclass.
- The Method in the super class is known as an *overridden* method which is called within a subclass.
- The version of the method defined by the superclass will be hidden.
- E.g. :

```
// Method overriding  
class A {  
int i ,j;  
A ( int a, int b) {  
i= a;  
j= b;  
}  
// display i and j  
void show ( ) {  
Sytem.out.println ( " i and j :  
" + i  
+ " " + j );  
}  
}  
class B extends A {  
int k;
```

```

B ( int a, int b, int c) {
super( a, b);
k= c;
}
// display k - this overrides
show( ) in A
void show ( ) {
System.out.println ( " k : " + k
);
}
}
class override {
public static void main ( String
args[ ] ) {
B subob = new B ( 1, 2, 3);
subob.show( ) ; // this calls
show( ) in B;
}
}

```

The output produced by this program is shown here :

k : 3

Constructors

- Performs automatic initialization of objects. ...
- Initializes an object immediately upon creation.
- Once defined, the constructor is automatically called immediately after the object is created, before the new operator completes.
- The default constructor automatically initializes all instance variables to zero and all object references to NULL.
- The default constructor is often sufficient for simple classes, but it usually won't do for more sophisticated ones.

- Once you define your own constructor, the default constructor is no longer used.

➤ Syntax:

- Have the same name with their class name.
- Have not return type, not even void.
- With curly braces

- **Constructor example:** [page 22\(Lab Manual\)](#)

- **Parameterized Constructors:** similar with parameterized methods. Refer your lab manual..

➤ Overloading Constructors:

Reding Assignment

- ❖ The this Keyword:
- ❖ Instance Variable Hiding:
- ❖ Garbage Collection:
- ❖ The finalize() Method:
- ❖ Overloading Constructors:
- ❖ Returning Objects:
- ❖ Recursion:
- ❖ Introducing Nested and Inner Classes:

Garbage Collection

- The ability of deallocation of memory automatically.
- When no references to an object exist, that object is assumed to be no longer needed, and the memory occupied by the object can be reclaimed.

- There is no explicit need to destroy objects as in C++.
- Garbage collection only occurs sporadically (if at all) during the execution of your program

Returning Objects

- A method can return any type of data, including class types that you create. For example:
- **Using objects as parameters:**
- **Recursion**
 - Recursion is the attribute that allows a method to call itself
 - A method that calls itself is said to be recursive E.g.

```
class Factorial { // A
    simple example of
    recursion.
    // this is a recursive
    method
    int fact(int n) {
    int result; if(n==1) return
    1;
    result = fact(n-1) * n;
    return result; } }
class Recursion {
    public static void
    main(String args[]) {
    Factorial f = new
    Factorial();
    System.out.println("Factori
    al of 3 is " + f.fact(3));
    System.out.println("Factori
    al of 4 is " + f.fact(4));
    System.out.println("Factori
    al of 5 is " + f.fact(5));
    } }
```

Nested and Inner Classes

- A class within another class known as nested classes.
- The scope of a nested class is bounded by the scope of its enclosing class.
- The inner classes can access instance variables of the outer class but not vice versa.
- There are two types of **nested classes**: *static* and *non-static*.
- **Static Class**: must access the members of its enclosing class through an object (Not directly). And are seldom used
- **None-Static Class**: Called inner class and has access to all of the variables and methods of its outer class directly in the same way that other non-static members of the outer class do.

Access Specifiers

Access level modifiers determine whether other classes can use a particular field or invoke a particular method.

There are *four* types access levels:

1. **No Modifier(Default)**: Visible to the package(source file), and it is called *package-private*
2. **Private**: Visible to the class only.
3. **Public**: Visible to the world.
4. **Protected**: Visible to the package and all subclasses.

Access Levels

Modifier	Class	Package	Subclass
public	Y	Y	Y
protected	Y	Y	Y
no modifier	Y	Y	N
private	Y	N	N

Static methods and Final Variables

- A static method can access only static data.
- A static method can call only other static data (*Static methods & variables*), but *can not call a non-static* data from it.
- Static method is belongs to the class and not to the object.
- A static method can be accessed directly by the class name and doesn't need any object.
- i.e ClassName.methodName(args)
- A static method cannot refer to "this" or "super" keywords in anyway
- Static methods are called without creating an object of class.
- They are referenced by the class name itself or reference to the Object of that class.
- They are stored in Permanent Generation space of heap (in Static memory)
- Static methods can not be overridden.

- But can be overloaded since they are resolved using *static binding* by compiler at compile time.
- **Note:** main method is static, since it must be accessible for an application to run, before any instantiation takes place.

// Example to illustrate Accessing the Static method(s) of the class.

```
class Monk{
    public static String
    monkName = "";
    public static void monk(String
    name){
        monkName = name; } }
class RealMonk {
    public static void main
    (String[] args)
        Monk.monk("Father
        Shinodah"); // Accessing the
        static method monk() and field
        by class name itself.
```

```
System.out.println(Monk.monkName
); //Accessing the static method
monk() by using Object's
reference.
```

```
Monk shinoda = new
Monk();
    shinoda.monk("is not
    dead");
```

```
System.out.println(shinoda.monkN
ame);    } }
```

```
//Output: Father Shinoda
        is not dead
```

Static methods and Final Variables...

- In Java, when final keyword is used with a variable of primitive data types (int, float, .. etc), value of the variable cannot be changed.
- E.g. public class Test {

```
public static void
main(String args[]) {
    final int i = 10;
    i = 30; // Error because
i is final.}}
```

Don't Confuse with Constants

- The java final keyword can be used in various context:
 1. **final variable** - A variable declared as final prevents the content of that variable being *modified*
 2. **final method** - A method declared as final prevents the user from *overriding* that method
 3. **final class** - A class declared as final cannot be extended thus prevents *inheritance*

Chapter – Three: Inheritance and Polymorphism

What is Inheritance?

- Inheritance is one of the cornerstones of object-oriented programming because it allows the creation of hierarchical classifications.
- **Inheritance** is the process where one class acquires the properties (methods and fields) of another.
- A class that is ***inherited*** is called a **superclass**.
- The class that ***does*** the ***inheriting*** is called a **subclass**.
- **OR:** The class which inherits the properties of other is known as subclass (derived class, child class) and

- The class whose properties are inherited is known as superclass (base class, parent class).
- Therefore, a subclass is a specialized version of a superclass.
- It inherits all of the instance variables and methods defined by the superclass and add its own, unique elements.
- **The extends keyword:** is the keyword used to inherit the properties of a class. Below given is the syntax of extends keyword.

➤ Syntax:

```
class Super{
.....}

class Sub extends Super{ }
```

```
class Calculation{
    int z;
    public void addition(int x,
int y){
        z=x+y;
        System.out.println("The
sum of the given numbers:"+z);
    }
    public void Substraction(int
x,int y){
        z=x-y;
        System.out.println("The
difference between the given
numbers:"+z);
    } }
```

```
public class My_Calculation
extends Calculation{

    public void
multiplication(int x, int y){
```

```

        z=x*y;
        System.out.println("The
product of the given
numbers:"+z);
    }
    public static void
main(String args[]){
    int a=20, b=10;
    My_Calculation demo = new
My_Calculation();
    demo.addition(a, b);
    demo.Substraction(a, b);
    demo.multiplication(a, b);

}}

```

Example 2

```

class A {
int i, j;
void showij() {
System.out.println("i and j: " +
i + " " + j); } }
class B extends A {
int k;
void showk() {
System.out.println("k: " + k); }
void sum() {
System.out.println("i+j+k: " +
(i+j+k)); } }
class SimpleInheritance {
public static void main(String
args[]) {
A superOb = new A();
B subOb = new B();
superOb.i = 10;
superOb.j = 20;
System.out.println("Contents of
superOb: ");
superOb.showij();
/* The subclass has access to
all public members of its
superclass. */
subOb.i = 7; subOb.j = 8;
subOb.k = 9;

```

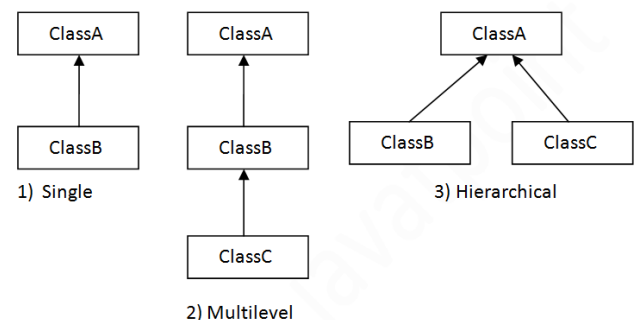
```

System.out.println("Contents of
subOb: ");
subOb.showij(); subOb.showk();
System.out.println();
System.out.println("Sum of i, j
and k in subOb:");
subOb.sum(); }}

```

Types of inheritance

1. **Simple/single inheritance:** is damn easy to understand. When a class extends another one class only.
2. **Multi-Level inheritance:** One can inherit from a derived class, thereby making this derived class the base class for the new class.
3. **Hierarchical inheritance:** One class is inherited by many sub classes.



- Java does not support the inheritance of multiple super classes into a single subclass. (Multiple & Hybrid inheritances)
- No class can be a superclass of itself
- Superclass can be used by itself.
- The superclass has no knowledge of what a subclass adds to it.
- Constructors are not members, so they are not inherited by subclasses.

- But the constructor of the superclass can be invoked from the subclass.

Member Access and Inheritance

- Although a subclass includes all of the members of its superclass, it cannot access those members of the superclass that have been declared as *private*.
- For example:

```
class A {
int i; private int j; // private
to A
void setij(int x, int y) {
i = x;
j = y;
}}
// A's j is not accessible here.
class B extends A {
int total;
void sum() {
total = i + j; // ERROR, j is
not accessible here
}}
class Access {
public static void main(String
args[]) {
B subOb = new B();
subOb.setij(10, 12);
subOb.sum();
System.out.println("Total is " +
subOb.total);
}}
```

A Superclass Variable Can Reference a Subclass Object

```
class Animal{
string name; int age
int show(){...}}
class Dog extends Animal{
String breed;
```

```
String barking(){
}}
//main{
Animal wild=new Animal();
Dog fries=new Dog();
wild=fries;
Wild.show();// correct show() is
defined in Animal
Wild.breed="Woo woo"// ERROR
}
```

The **super** keyword

- Since encapsulation is a primary attribute of OOP, Whenever a subclass needs to refer to its immediate superclass, it can do so by use of the keyword **super**.
- **Super** has two general forms.
 1. Used to access parent class instance members.
 2. Used to access parent class constructor.
- 1. **Used to access parent class instance members.**
 - **super** acts somewhat like *this*, except that it always refers to the superclass of the subclass in which it is used.
 - **Super** has the form: **super.member** Here, member can be either a **method** or an **instance** variable.
 - **Super** is most applicable to resolve member hiding.
 - **super()** must always be the first statement executed inside a subclass' constructor.
 - Consider the next simple class

1.1.Used to access parent class instance members-Variable.

```
class Animal{
String color="white";
}
class Dog extends Animal{
String color="black";
void printColor(){
System.out.println(color);//prints color of Dog class
System.out.println(super.color);//prints color of Animal class
}
}
class TestSuper1{
public static void main(String args[]){
Dog d=new Dog();
d.printColor();
}}
```

Output:
black
white

1.2.Used to access parent class instance members-Method.

```
class Animal{
void eat(){System.out.println("eating...");}
}
class Dog extends Animal{
void eat(){System.out.println("eating bread...");}
void bark(){System.out.println("barking...");}
void work(){
super.eat();
bark();
}
}
class TestSuper2{
public static void main(String args[]){
Dog d=new Dog();
d.work();
}}
```

Here super is used to resolve overridden methods.

Output:
eating ...
barking...

```
// Using super to overcome name hiding.
class A {
int i; }
class B extends A {
int i; // this i hides the i in A
B(int a, int b) {
super.i = a; // i in A
i = b; // i in B }
void show() {
System.out.println("i in superclass: " + super.i);
System.out.println("i in subclass: " + i);
}}
class UseSuper {
public static void main(String args[]) {
B subOb = new B(1, 2);
subOb.show();
}}
```

2. Used to access parent class constructor.

- A subclass can call a constructor defined by its superclass by use of the following form of super: **super(arg-list);**
- Here, arg-list specifies any arguments needed by the constructor in the superclass.
- super() must always be the first statement executed inside a subclass' constructor.

```

class Animal{
Animal(){System.out.println("animal is created");}
}
class Dog extends Animal{
Dog(){
super();
System.out.println("dog is created");
}
}
class TestSuper3{
public static void main(String args[]){
Dog d=new Dog();
}}

```

Output:

animal is created
dog is created

If you need more: **Search** on

<https://www.javatpoint.com/super-keyword>

Creating a Multi-level Hierarchy

```

class SuperClass3
{
    public void display3()
    {
System.out.println("This is
display3 method");
    } }
class SuperClass2 extends
SuperClass3
{
    public void display2()
    {
System.out.println("This is
display2 method");
    } }
class SuperClass1 extends
SuperClass2
{
    public void display1()
    {

```

```

System.out.println("This is
display1 method");
    } }
class MultiLevelDemo
extends SuperClass1
{
    public static void
main(String args[])
    {
        MultiLevelDemo
object = new MultiLevelDemo();
        object.display1();
        object.display2();
        object.display3();
    } }

```

When Constructors Are Called?

- In a class hierarchy, constructors are called in order of derivation, from superclass to subclass.
- Further, since super() must be the first statement executed in a subclass' constructor, this order is the same whether or not super() is used.
- If super() is not used, then the default or parameter less constructor of each superclass will be executed.

Why?

```

// Demonstrate when constructors
are called. Create a super
class.
class A {
A() {
System.out.println("Inside A's
constructor.");
} }
// Create a subclass by
extending class A.
class B extends A {
B() {
System.out.println("Inside B's
constructor.");
} }

```

```

}}
// Create another subclass by
extending B.
class C extends B {
C() {
System.out.println("Inside C's
constructor.");
}}
class CallingCons {
public static void main(String
args[]) {
C c = new C();
}}

```

What is the output of this program? And How

Output:

Inside A's Constructor

Inside A's Constructor

Inside A's Constructor

Method Overriding

- In a class hierarchy, when a method in a subclass has the same name and type signature as a method in its superclass, then the method in the subclass is said to override the method in the superclass.
- When an overridden method is called from within a subclass, it will always refer to the version of that method defined by the subclass.
- The version of the method defined by the superclass will be hidden.
- Method overriding occurs only when the names and the type signatures of the two methods are identical.

What is the output of the following program?

// Method overriding.

```

class A {
int i, j;
A(int a, int b) {

```

```

i = a;
j = b; }
// display i and j
void show() {
System.out.println("i and j: " +
i + " " + j);
}}
class B extends A {
int k;
B(int a, int b, int c) {
super(a, b);
k = c; }
// display k - this overrides
show() in A
void show() {
System.out.println("k: " + k);
}}
class Override {
public static void main(String
args[]) {
B subOb = new B(1, 2, 3);
subOb.show(); Which method is
called? // this calls show() in
B
}}

```

Output:

K: 3

```

class A {
int i, j;
A(int a, int b) {
i = a;
j = b; }
void show() {
System.out.println("i and j: " +
i + " " + j);
}}
class B extends A {
int k;
B(int a, int b, int c) {
super(a, b);
k = c; }
void show() {
super.show(); // this calls A's
show()

```

```

System.out.println("k: " + k);
}}
class Override {
public static void main(String
args[]) {
B subOb = new B(1, 2, 3);
subOb.show(); Which one is
called? // this calls show() in
B
}}

```

Output:

I and j: 1 2
K: 3

Rules of method overriding in Java..

- **Argument list:** The argument list of overriding (method of subclass) and Overridden (method of super class) method must be similar.
- The **data types** of the arguments and their sequence should exactly match.
- **Method names:** must be similar.
- private, static and final methods cannot be overridden as they are local to the class.
- **Access Modifier** of the overriding method cannot be more restrictive than the overridden method of parent class.
- For example look the following Program.

```

// The following program cannot
be run well.
class MyBaseClass{
    public void disp()
    {

System.out.println("Parent class
method");
    }}

```

```

class MyChildClass extends
MyBaseClass{
    protected void disp(){
        System.out.println("Child
class method");
    }
    public static void main(
String args[]) {
        MyChildClass obj = new
MyChildClass();
        obj.disp();
    }}

```

Output:

Exception in thread "main"
java.lang.Error: Unresolved
compilation
problem: Cannot reduce the
visibility of the inherited
method from MyBaseClass

// The following program can run
very well. But what is the
output?

```

class MyBaseClass{
    protected void disp()
    {

System.out.println("Parent class
method");
    }}
class MyChildClass extends
MyBaseClass{
    public void disp(){
        System.out.println("Child
class method");
    }
    public static void main(
String args[]) {
        MyChildClass obj = new
MyChildClass();
        obj.disp();
    }}

```

Output:

Child class method

Why Overridden Methods?

- The ability to define a behavior that's specific to the subclass type, which means a subclass can implement a parent class method based on its requirement.
- The class can give its own specific implementation to an inherited method without even modifying the parent class code.
- This is helpful when a class has several child classes, so if a child class needs to use the parent class method, it can use it and the other classes that want to have different implementation can use overriding feature to make changes without touching the parent class code.
- Method overriding is used for runtime polymorphism

Method Overriding

Dynamic Method Dispatch

- Dynamic method dispatch is the mechanism by which a call to an overridden method is resolved at run time, rather than compile time.
- Dynamic method dispatch is important because this is how Java implements run-time polymorphism
- When an overridden method is called through a superclass reference, Java determines which version of that method to execute based upon the type of the object being referred to at the time the call occurs.
- Thus, this determination is made at run time.
- Method Overriding is an example of runtime polymorphism.

- When a parent class reference points to the child class object then the call to the overridden method is determined at runtime, because during method call which method (parent class or child class) is to be executed is determined by the type of object.
- This process in which call to the overridden method is resolved at runtime is known as dynamic method dispatch.
- Lets see an example to understand this:

```
class ABC{
    //Overridden method
    public void disp()
    {
        System.out.println("disp()
method of parent class");
    }}
class Demo extends ABC{
    //Overriding method
    public void disp(){
        System.out.println("disp()
method of Child class");
    }
    public void newMethod(){
        System.out.println("new
method of child class");
    }
    public static void main(
String args[]) {
        /* When Parent class
reference refers to the parent
class object then in this
case overridden method (the
method of parent class) is
called. */
        ABC obj = new ABC();
        obj.disp();
    }
}
```

```

/* When parent class reference
refers to the child class object
then the overriding method
(method of child class) is
called. This is called dynamic
method dispatch and runtime
polymorphism*/
    ABC obj2 = new Demo();
    obj2.disp();
}
}

```

Example 2

```

// Dynamic Method Dispatch.
class A {
void callme() {
System.out.println("Inside A's
callme method");
}}
class B extends A {
// override callme()
void callme() {
System.out.println("Inside B's
callme method");
}}
class C extends A {
// override callme()
void callme() {
System.out.println("Inside C's
callme method");
}}
class Dispatch {
public static void main(String
args[]) {
A a = new A(); // object of type
A
B b = new B(); // object of type
B
C c = new C(); // object of type
C
A r; // obtain a reference of
type A
r = a; // r refers to an A
object

```

```

r.callme(); // calls A's version
of callme
r = b; // r refers to a B object
r.callme(); // calls B's version
of callme
r = c; // r refers to a C object
r.callme(); // calls C's version
of callme
}}

```

Output:

```

Inside A's callme method
Inside B's callme method
Inside C's callme method

```

Abstract Classes

- A classes declared with abstract keyword and can't be instantiated.
- Used to hide meaningless methods:
- There can be no objects of an abstract class. ...
- An abstract class cannot be directly instantiated with the **new** operator.
- *You cannot declare abstract constructors, or abstract static methods*
- **Syntax:** abstract class *class_name*

Example:

```

abstract class A {
abstract void callme();
// concrete methods are still
allowed in abstract classes
void callmetoo() {
System.out.println("This is a
concrete method.");
}}
class B extends A {
void callme() {
System.out.println("B's
implementation of callme.");
}}
class AbstractDemo {
public static void main(String
args[]) {

```

```
B b = new B();
b.callme();
b.callmetoo();
A ob=new A();// ERROR }}
```

Using *final* with Inheritance

➤ The keyword **final** has three uses.

1. To create the equivalent of a named **constant (final variable)** - A variable declared as final prevents the content of that variable being *modified*.
2. To Prevent Overriding in **Methods (final method)** - A method declared as final cannot be *overridden*.

E.g.

```
class A {
    final void meth() {
        System.out.println("This
        is a final method.");
    }
}
class B extends A {
    void meth() { // ERROR!
        Can't override.
        System.out.println("Ille
        gal!");
    }
}
```

3. To Prevent Inheritance in **Classes (final class)** - A class declared as final cannot be extended thus prevents *inheritance*. It is illegal to declare a class as both *abstract* and *final*. E.g.

```
final class A {
    // ...}
    // The following class is
    illegal.
    class B extends A { //
    ERROR! Can't has subclass A
    // ...}
```

Binding & final

- Binding of overridden methods happen at runtime which is known as *dynamic binding*.
- The compiler is free to inline calls to them because it “knows” they will not be overridden by a subclass.
- Normally, Java resolves calls to methods *dynamically, at run time* called *late binding*.
- However, since final methods cannot be overridden, a call to one can be resolved *at compile time* called *early binding*.
- Static Binding:
- Dynamic Binding:
- **Multi-threaded** program contains two or more parts that can run concurrently.
- Each part can handle different task at the same time making optimal use of the available resources. Especially, when your computer has multiple CPUs.

Summery:

Abstract:

- **Classes:** declared with abstract keyword and *can't be instantiated*.
- **Methods:** without body (no implementation)

Final:

- **Class:** declared as final prevents *inheritance*.
- **Method:** declared as final prevents *overriding* that method.
- **Variable:** declared as final prevents the content *modification*.

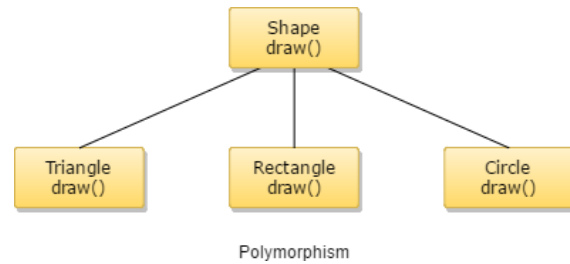
Static:

- **Method:** can call only other static data, belongs to class, Can't b override.
- **Variable:** declared with the static keyword in a class, but outside a method, constructor or a block.

Polymorphism

- The dictionary definition of polymorphism refers to a principle in biology in which an organism or species can have many different forms or stages.
- This principle can also be applied to object-oriented programming like the Java language
- Polymorphism is the ability of an object to take on many forms.
- The most common use of polymorphism in OOP occurs when a parent class reference is used to refer to a child class object.
- Any Java object that can pass *more than one* IS-A test is considered to be polymorphic.
- In Java, all Java objects are polymorphic since any object will pass the IS-A test for their own type and for the class Object.
- Polymorphism is derived from 2 Greek words: poly and morphs.
- The word "poly" means many and "morphs" means forms. So polymorphism means many forms.
- polymorphism is the capability of an action or method to do different things based on the object that it is acting upon.

- It is one of the basic principles of object oriented programming.



Types of Polymorphism

- **There are 2 basic types of polymorphism:**
 1. Static Polymorphism
 2. Dynamic Polymorphism.
- Some programmers classify polymorphism in to three:
 1. Ad-hoc (overloading and overriding),
 2. Parametric (generics) and
 3. Dynamic method binding.
- 1. Static polymorphism**
 - Static Polymorphism is in other words termed as *compile-time binding* or *early binding*.
 - Static binding occurs at compile time.
 - Method overloading is a case of static binding and in this case binding of method call to its definition happens at the time of compilation.
- 2. Dynamic polymorphism**
 - Dynamic (or late) method binding is the ability of a program to resolve references to *subclass* methods at runtime.
 - For example assume that three subclasses (Cow, Dog and Snake) have been created based on the Animal

abstract class, each having their own speak() method.

- Although each method reference is to an Animal (but no animal objects exist), the program will resolve the correct method reference at runtime.

```
public class AnimalReference
{
    public static void main(String
args[])
    {
        Animal ref ;           //
set up var for an Animal
        Cow aCow = new Cow("Bossy");
// makes specific objects
        Dog aDog = new Dog("Rover");
        Snake aSnake = new
Snake("Ernie");
        // now reference each as an
Animal
        ref = aCow; ref.speak();
        ref = aDog; ref.speak();
        ref = aSnake; ref.speak();
    }
}
```

The instance of operator

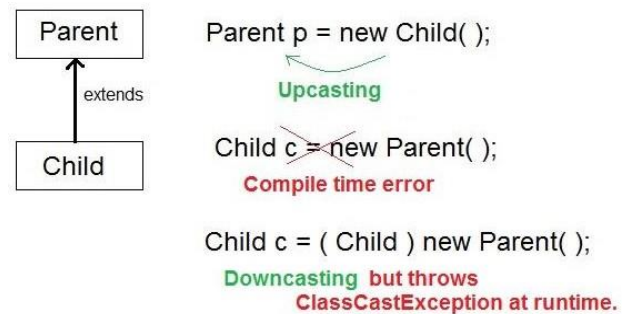
- In Java, instanceof operator is used to check the type of an object at runtime.
- It is the means by which your program can obtain run-time type information about an object.
- instanceof operator is also important in case of casting object at runtime.
- instanceof operator return boolean value, if an object reference is of specified type then it return true otherwise false.

```
public class Test
{
    public static void
main(String[] args)
    {
```

```
Test t= new Test();
System.out.println(t
instanceof Test);
}}
```

Output: True

The instance of operator-Down Casting



E.g. of downcasting with instanceof operator

```
class Parent{ }
public class Child extends
Parent{
    public void check(){

System.out.println("Sucessfull
Casting");
    }
    public static void
show(Parent p){
        if(p instanceof
Child){
            Child
b1=(Child)p;
            b1.check();
        }
    }
    public static void
main(String[] args){
        Parent p=new Child();
        Child.show(p);
    }} Output: Sucessfull
```

Casting

How polymorphism supported in java

- Java has excellent support of polymorphism in terms of *Inheritance*, method *overloading* and method *overriding*.
- Method overriding allows Java to invoke method based on a particular object at run-time instead of declared type while coding.

Where to use polymorphism in code?

1. Method argument:
2. Variable names:
3. Return type of method:

Chapter – Four: Exception Handling

- “a person or thing that is excluded from a general statement or does not follow a rule.”
- An exception is an event, which occurs during the execution of a program, that disrupts the normal flow of the program's instructions.
- An exception is an indication of a problem that occurs during a program's execution.
- An exception is a runtime error.
- A Java exception is an instance of a class derived from Throwable.
- The Throwable class is contained in the java.lang package, and subclasses of Throwable are contained in various packages.
- Errors related to GUI components are included in the java.awt package; numeric exceptions are included in the java.lang package because they are related to the java.lang.Number class.

- You can create your own exception classes by extending Throwable or a subclass of Throwable.
- **Exceptions occur for various reasons. E.g.**
 - A user has entered invalid data.
 - A file that needs to be opened cannot be found.
 - A network connection has been lost in the middle of communications or the JVM has run out of memory.
- Some of these exceptions are caused by:
 - User error,
 - Programmer error,
 - Physical resources that have failed in some manner.

```
import java.util.Scanner;
import java.util.*;
public class ExceptionDemo {
    public static void
main(String args[])
    {
        Scanner reader=new
Scanner (System. in);
```

```
System.out.println("Enter a
number :");
        int num=
reader.nextInt(); // try to
enter your name
```

```
System.out.println("The number
is :"+ num); } }
```

Stack trace: an Information about an exception

Catching Exception...

```
import java.util.Scanner;
import java.util.*;
public class ExceptionDemo {
```

```

    public static void
main(String args[]){
    Scanner reader=new
Scanner(System.in);

System.out.println("Enter a
number :");

        try {
            int num=
reader.nextInt();

System.out.println("The number
is : " + num);
        }
        catch
(InputMismatchException e){

System.out.println("There is
type mis match :");
        }

        finally{

System.out.println("This part is
always done");
        } } }

```

Exception Types

✓ There two categories of Exceptions:

1. **Checked exception:** Is an exception that occurs at the compile time,
 - ✓ These are also called as compile time exceptions.
 - ✓ These exceptions cannot simply be ignored at the time of compilation,
 - ✓ The Programmer should take care of (handle) these exceptions.
 - ✓ For example, if you use FileReader class in your program to read data from a file, if the file specified in its constructor

doesn't exist, then a FileNotFoundException occurs, and compiler prompts the programmer to handle the exception.

```

import java.io.File;
import java.io.FileReader;
public class FileNotFound_Demo {
    public static void
main(String args[]){
    File file=new
File("E://file.txt");
    FileReader fr = new
FileReader(file);
    }}

```

➤ If you try to compile the above program you will get exceptions like this.

```

C:\>javac FileNotFound_Demo.java
FileNotFound_Demo.java:8: error:
unreported exception
FileNotFoundException; must be
caught or declared to be thrown
    FileReader fr = new
FileReader(file);
1 error

```

- ✓ **Unchecked exceptions:** Is an exception that occurs at the time of execution, these are also called as Runtime Exceptions.
 - ✓ These include programming bugs, such as logic errors or improper use of an API.
 - ✓ Runtime exceptions are ignored at the time of compilation.
 - ✓ For example, if you have declared an array of size 5 in your program, and trying to call the 6th element of the array then an ArrayIndexOutOfBoundsException occurs.

2. **Other:** Errors

Exception Vs Errors

1. **Errors:** An Error indicates *serious problem* that a reasonable application should not try to catch.
2. **Exception:** Exception indicates conditions that a reasonable application might try to catch.

Using try and catch...

General form of try/catch:

```
Try{  
  //block of code to monitor for  
  errors  
}  
Catch(ExceptionType1 exOb ) {  
  //handler for ExceptionType1  
}  
Catch(ExceptionType2 exOb ) {  
  //handler for ExceptionType2  
}
```

File and Stream

- A file is an object on a computer that stores data, information, settings, or commands used with a computer program.
- E.g. Image, text, video, audio...(Multimedia)
- In a graphical user interface (GUI) such as Microsoft Windows, files display as icons that relate to the program that opens the file.
- A stream is an abstraction that either produces or consumes information.
- A stream is a communication channel that a program has with the outside world. It is used to transfer data items in succession.

- A stream is linked to a physical device by the Java I/O system.
- Java implements streams within class hierarchies defined in the java.io package.
- there are **two kinds of Streams**
- **InPutStream:** The InputStream is used to read data from a source.
- **OutPutStream:** the OutputStream is used for writing data to a destination.

1. Byte Streams:

- Java byte streams are used to perform input and output of 8-bit bytes.
- A byte stream is suitable for processing raw data like binary files.
- The most frequently used classes are **FileInputStream** and **FileOutputStream**.

```
import java.io.*;  
public class CopyFile {  
    public static void  
    main(String args[]) throws  
    IOException  
    {  
        FileInputStream in = null;  
        FileOutputStream out =  
        null;  
        try {  
            in = new  
            FileInputStream("C:\\Users\\FIKR  
            TE\\Documents\\text.txt");  
            out = new  
            FileOutputStream("C:\\Users\\FIK  
            RTE\\Documents\\textCopied.txt")  
            ;  
            int c;  
            while ((c = in.read())  
            != -1) {
```

```

        out.write(c); }
    }finally {
        if (in != null) {
            in.close(); }
        if (out != null) {
            out.close(); } } }

```

2. Character Streams:

- Java Character streams are used to perform input and output for 16-bit unicode.
- Character stream is useful when we want to process text files.
- These text files can be processed character by character.
- The most frequently used classes are , *FileReader* and *FileWriter*.

```

import java.io.*;
public class CopyFile {
    public static void
main(String args[]) throws
IOException
{
    FileReader in = null;
    FileWriter out = null;
    try {
        in = new
FileReader("input.txt");
        out = new
FileWriter("output.txt");
        int c;
        while ((c = in.read())
!= -1) {
            out.write(c); }
        }finally {
            if (in != null) {
                in.close(); }
            if (out != null) {
                out.close();
            } } } }

```

- **Reading and Writing Files:** As described earlier, A stream can be defined as a sequence of data.

- The *InputStream* is used to read data from a source and the *OutputStream* is used for writing data to a destination.

Chapter – Five: Java Interface

- An interface in java is a blueprint of a class.
- It has static constants and abstract methods.
- The interface in java is a mechanism to achieve abstraction.
- There can be only abstract methods in the java interface not method body (contain only signature, not body).
- It is used to achieve abstraction and multiple inheritance in Java.
- Java Interface also represents IS-A relationship.
- It cannot be instantiated just like abstract class.
- There are mainly three reasons to use interface.
 - It is used to achieve abstraction.
 - By interface, we can support the functionality of multiple inheritance.
 - It can be used to achieve loose coupling.
- Interface is declared by using ***interface*** keyword.

Syntax:

```
interface <interface_name>{
```

```
// declare constant fields // declare methods
that abstract // by default.
```

```
}
```

```
interface <interface_name>{
```

```
// declare constant fields // declare methods
that abstract // by default.
```

```
}
```

- It provides total abstraction;
- Means all the methods in interface are declared with empty body and are *public* and all fields are public, static and final by default.
- A class that implement interface must implement all the methods declared in the interface.
- If a class implements an interface and does not provide method bodies for all functions specified in the interface, then class must be declared abstract.

Rules of Interface

- Variables declared in an interface are public, static & final by default.
- Java does not allow you to extend more than one class, However you can implement more than one interfaces in your class.
- class implements interface but an interface extends another interfaces.

Implementing Interface...E.g.

```
interface MyInterface
{
    /* All the methods are public
    abstract by default
    * As you see they have no
    body
```

```
*/
```

```
    public void method1();
    public void method2(); }
```

Example 2:

```
interface MyInterface{
    /* compiler will treat them
    as: public abstract void
    method1(); public abstract void
    method2(); */
    public void method1();
    public void method2(); }
class Demo implements
MyInterface{
    /* This class must have to
    implement both the abstract
    methods else you will get
    compilation error */
    public void method1(){
        System.out.println("imple
mentation of method1");
    }
    public void method2(){
        System.out.println("imple
mentation of method2");
    }
    public static void
    main(String arg[]){
        MyInterface obj = new
        Demo();
        obj.method1();}}
```

Output:implementation of method1

Extending Interface

- It is called Interface inheritance

```
interface Printable{
    void print(); }
interface Showable extends
Printable{
    void show(); }
class TestInterface4 implements
Showable{
    public void
    print(){System.out.println("Hell
o");}
```

```

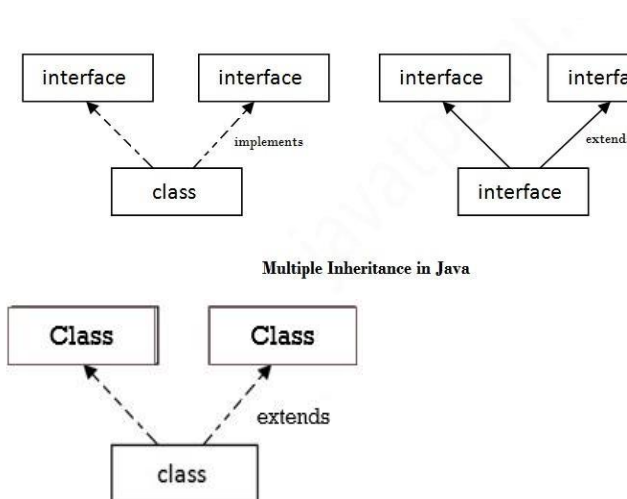
public void
show(){System.out.println("Welco
me");}
public static void main(String
args[]){
TestInterface4 obj = new
TestInterface4();
obj.print();
obj.show();
}}

```

Extending multiple Interface

Multiple inheritance in Java by interface

- If a class implements multiple interfaces, or an interface extends multiple interfaces i.e. known as multiple inheritance.



Extending multiple Interface

- A single interface can extend multiple interfaces, below is a simple example.

```

public interface InterfaceA {
    public void
doSomething();
}
public interface InterfaceB {

```

```

    public void
doSomething();
}
public interface InterfaceC
extends InterfaceA, InterfaceB {
    //same method is declared in
    both InterfaceA and InterfaceB
    public void
doSomething();
}

```

E.g.

```

public interface InterfaceA {
    public void
doSomething();
}
public interface InterfaceB {
    public void
doSomething();
}
public interface InterfaceC
extends InterfaceA, InterfaceB {
    //same method is declared in
    both InterfaceA and InterfaceB
    public void
doSomething();
}

```

Implementing multiple interface

```

interface Printable{
void print();
}
interface Showable{
void show();
}
class A7 implements Printable,Sh
owable{
public void print(){System.out.p
rintln("Hello");}
public void show(){System.out.pr
intln("Welcome");}
public static void main(String a
rgs[]){
A7 obj = new A7();

```

```
obj.print();  
obj.show();  
} }
```

Tagging Interface

- An empty interface is known as tag or marker interface.
- For example Serializable, EventListener, there are few other tag interfaces as well.
- These interfaces do not have any field and methods in it.
- You must be thinking if they are empty why class implements them? What's the use of it?
- Class implements them to claim the membership in a particular set.
- Basically, Tag interfaces are meaningful to the JVM
- It would improve the readability of your code.

The similarity b/n Interface & Class

- An interface can contain any number of method and fields.
- It is written in a file with a .java extension. With the name of the interface matching the name of the file.
- The byte code of an interface appears in a .class file.

The d/c b/n Interface & Class

- You cannot instantiate an interface
- Interface has not constructor
- Methods in an interface are all abstract.
- Fields in an interface are all static and final
- An interface can extend multiple interfaces

Summery

- By default, methods in interface are public and abstract.
- Data members are public, static and final.
- An interface with empty body is called Tag/marker interface.

